**Algorithms – Final Exam**

3.3.24 – term A

Prof. Tami Tamir

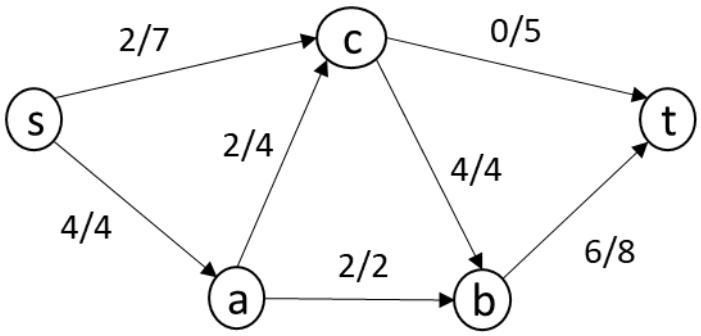
- Exam length: 3 hours
- Allowed open material: Attached fact-sheets + one paper brought by the student.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

G O O D  L U C K !

## ♣ Question 1 (20 pts.)

♣ **1.1** (**10 pts**.) The figure presents a flow network with a legal flow. Every edge e∈E is labeled by f(e)/c(e) . It is known that the given flow is a result of **two** iterations of Ford & Fulkerson algorithm.

**No need to justify your answers in this section.**




1. What was the augmenting path in the first iteration?

   s-a-c-b-t

2. What was the augmenting path in the second iteration? s-c-a-b-t

3. Suggest an augmenting path for the next (third) iteration: s-c-t

4. What is the value of the maximum flow in the network? 11

5. Write all sets of nodes that define a min-cut in the network:

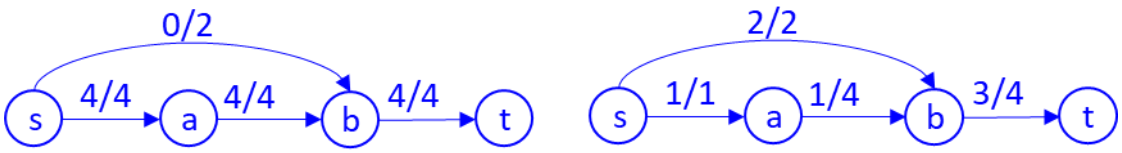
   {s}, {s,a,c}

**1.2** (**2*5=10 pts**). Let G=(V,E) be a flow network. For a given edge $e_1 \in E$, it is known that if the capacity of $e_1$ is decreased by 3, then the value of the maximum flow in the network decreases by 1.

Prove or refute each of the following claims (separately).

♣ a. Before the capacity change, in every max-flow, $f(e_1) < c(e_1)$.

False. Let $e_1$=(s,a). In the left network, the max-flow value is 4, and $e_1$ is saturated.
In the right network, the capacity of $e_1$ reduces by 3, and the max-flow value is 3.




♣ b. After the capacity change, in every max-flow, $f(e_1) = c(e_1)$.

True. Since the value of the max-flow reduces, it must be that $e_1$ is in every min-cut of the new network, and therefore, must be saturated.

Note: An edge is downward critical iff reducing its capacity by any value (even by ε) results in reduced max-flow. The "even by ε" is important. For example, the edge (sa) in the figure above is not downward critical – if its capacity is reduced by x≤2, then the max-flow is not hurt. Therefore, part b cannot be justified 'since $e_1$ is downward critical'.



## ♣ Question 2 (14 pts.)

n girls arrive at a hall where a performance is held, and are interested in buying tickets. The i-th girl wants to buy $t_i$ tickets. The price of tickets increases as time goes by. The basic price for one ticket is 10 NIS. The girls have to stand in line, and it is known that the k-th girl in line will pay for each ticket (10+k) NIS (regardless of the number of tickets she buys).
If, for example, n=3, $t_1$=5, $t_2$=3, $t_3$=6 and they stand in line according to their indices, then the total price they will pay will be $5 \cdot (10 + 1) + 3 \cdot (10 + 2) + 6 \cdot (10 + 3)$.

Suggest **a greedy algorithm** that determines the order in which the girls should stand in line so that the total price they pay together is as low as possible.

1. **(4 pts.)** Describe the algorithm, and specify its time complexity.
2. **(10 pts.)** State and prove the greedy choice property.

Algorithm: Sort the girls such that $t_1 \geq t_2 \geq .. \geq t_n$.
The order in the line is by the sorting.

Time complexity: O(n log n).

GCP: There exists an optimal order in which girl #1 (alternatively, a girl with maximal $t_i$) is first.

Proof. Let A be any order, and assume that the first girl is girl $a \neq 1$, and that girl #1 is in the k-th position.
By switching the positions of these two girls, the payments of the other girls do not change.
The total payments of girl 1 and girl a before the swap is $t_a(10+1)+t_1(10+k) = 10t_a + 10t_1 + t_a+kt_1$.
The total payments of girl 1 and girl a after the swap is $t_a(10+k)+t_1(10+1) = 10t_a + 10t_1 + kt_a+t_1$.

Since k>1 and, by the sorting, $t_1 \geq t_a$, we have that $(k-1)t_a \leq (k-1)t_1$. Therefore, $kt_a+t_1 \leq t_a+kt_1$, implying that the swap can only reduce the total payments. We conclude that A can be modified to agree with the algorithm without hurting the total payment.



**Question 3 (18 pts.)**

Bob the Minion reaches a path consisting of n tiles. For all $1 \le i \le n$, a bunch of $b_i$ bananas is placed in tile i. All $b_i$ values can be assumed to be even numbers greater than 16.
Bob can collect bananas from the bunches in their order according to the following rules:
From each bunch $1 \le i \le n$, Bob can take all the bananas and retire (leave the path), or take half of the bananas and skip the next tile (do not take anything from bunch i+1), or take eight bananas and move to the next tile.
If, for example, n=7 and the bunches are of sizes 24,26,60,70,32,40,26, then Bob can collect 8+8+8+8+8+8+26 or 8+13+0+ 35+0+40 or 12+0+8+70 bananas (there may be additional series).

Suggest an algorithm based on **dynamic programming** that returns the maximum number of bananas Bob can collect. For each $1 \le i \le n$, let $P_{all}[i]$ denote the maximum number of bananas that Bob can collect from the first i tiles, assuming that he takes all the bananas in bunch i. Let $P_{half}[i]$ denote the maximum number of bananas that Bob can collect from the first i tiles, assuming that he takes half of the bananas in bunch i, and let $P_8[i]$ denote the maximum number of bananas that Bob can collect from the first i tiles, assuming that he takes 8 bananas from bunch i.

**3.1** (2 pts.) What is the expression describing the solution's value (no need to explain)?

MAX $_{1 \le k \le n}$ $P_{all}[k]$

♣ **3.2** (12 pts.) Write a dynamic programming formula for calculating $P_{all}$, $P_{half}$, $P_8$. Distinguish between the base and the general case. Explain the formula in words.

Base case: for i=0, $P_8[0]=P_{half}[0]=0$.

for i=1, $P_{all}[1]=b_1$, $P_8[1]=8$ $P_{half}[1]=b_1/2$

General case for i>1
$P_{all}[i]= b_i + \max(P_8[i-1], P_{half}[i-2])$
$P_8[i]= 8 + \max(P_8[i-1], P_{half}[i-2])$
$P_{half}[i]= b_i/2 + \max(P_8[i-1], P_{half}[i-2])$

(it is possible not to define i=0 in the base case, and have a different formula for i=2. Specifically, $P_{all}[2]= b_2 + 8$, $P_8[2]= 16$, $P_{half}[2]= b_2/2 + 8$. It is also possible to define $P_{half}[-1]=0$, the entries of i=0 as above, and start the general case from i=1).

Explanation: In every tile, we take the amount of bananas that fit the table description, and add the maximal number of bananas that could be collected from previous tiles, distinguishing between the two options to reach tile i.

♣ **3.3** (4 pts.) Describe the DP-table, its size, and the order it is filled. Analyze the time-complexity of the algorithm as a function of n. Justify your answer.

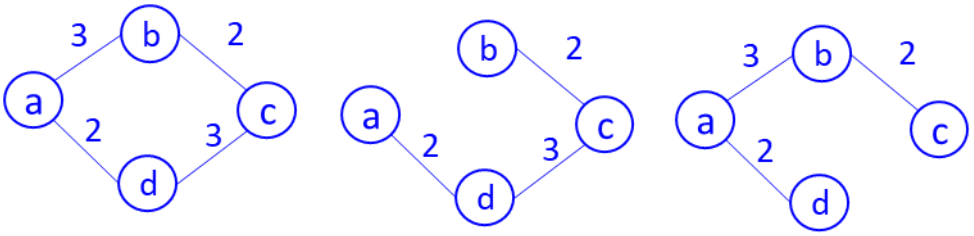
The algorithm fills three 1-dim tables of size n (or a single n*3 table).
For all i, all three entries of index i (in the 3 tables) are filled before moving to index i+1.
Each entry requires $\Theta(1)$ time, therefore the total time complexity is $\Theta(n)$.
Computing the maximal entry in $P_{all}$, requires additional $\Theta(n)$ [no points were reduced for ignoring this].

**♣ Question 4 (3*6=18 pts.)**

Let $G = (V, E)$ be a simple undirected graph, with positive edge costs. Let $T$ be a given **minimum spanning tree** of $G$. Given two nodes $a, b \in V$, let $p$ be the path in $T$ that connects $a$ and $b$.

**♣ 4.1** Denote by $mincost(p)$ the cost of the cheapest edge in $p$.
**Prove or refute**: In every MST of $G$, the cheapest edge in the path connecting $a$ and $b$ is of cost $mincost(p)$.

Counter example: For the graph G in the left figure, and T=middle figure, p=a-d-c-b, and $mincost(p) = 2$. The graph has another MST (right figure) in which the cheapest edge in the path connecting a and b is 3.




**♣ 4.2** Denote by $maxcost(p)$ the cost of the most expensive edge in $p$.
**Prove or refute**: In every MST of $G$, the most expensive edge in the path connecting $a$ and $b$ is of cost $maxcost(p)$.

Proof: Assume by contradiction that there exists an MST T' in which a and b are connected by a path p', such that $maxcost(p) \neq maxcost(p')$.
Assume w.l.o.g., that $maxcost(p) < maxcost(p')$.
The definition of $maxcost(p)$ implies there exists a path in G connecting a and b consisting of edges of cost at most $maxcost(p)$.
By the claim proven in class, in every MST, a and b are connected by a path in which all edges have cost at most $maxcost(p)$, contradicting the fact that $maxcost(p')$ is higher.

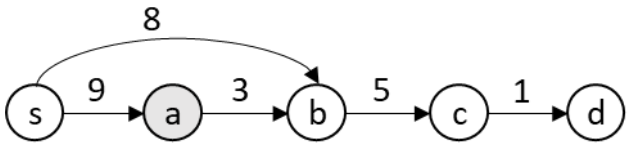
♣ **4.3** It is given that $T$ is the output of $Prim$ algorithm, in a run that begins at node $a$.
Denote $p = a - \cdots - u - b$ (that is, $u$ is the last node before $b$ in $p$, possibly, $u = a$) .
**Prove or refute:** The edge $(u, b)$ is the last among the edges of $p$ to join the MST $T$.

Proof: In a run of Prim, the edges added to T form a single connected component. Since the run begins at 'a', it must be that 'a' belongs to this component. If the last edge in p that is added to T is not (u,b), then 'b' is already in the connected component, that is, there is already a path between a and b in T before the last edge in p is added. Therefore, the addition of the last edge closes a cycle. A contradiction to the fact that Prim returns a tree.

♣ **Question 5 (18 pts.)**

Given is a directed graph $G = (V, E)$, a node $s \in V$, and a node $a \in V$. Let $c: E \to \mathbb{R}^+$ be a positive cost function defined for the graph edges.
The following payment method is applied: If a path passes through node $a$, then the cost of passing through the edge that leaves $a$ is 1, the cost of passing through the edge that follows it on the path is 2, and so on (the cost of passing through the k-th edge after passing through node $a$ is k). Until the pass through $a$, or in a path that does not pass in $a$ at all, the costs are according to the given cost function $c$.
Denote by $cost(s, v)$ the cost of a cheapest path from $s$ to $v$ according to the above payment method. Suggest an efficient algorithm that finds $cost(s, v)$ for each node $v \in V$. Describe the algorithm in words and formally, justify its correctness (write the main claim and justify it shortly), and analyze its time complexity.

**Example**: In the graph in the figure, the cost of the path s-a-b-c is 9+1+2=12, the cost of the path s-b-c is 8+5=13, therefore cost(s,c)=12. The cost of the path s-a-b-c-d is 9+1+2+3=15, the cost of the path s-b-c-d is 8+5+1=14, therefore cost(s,d)=14.




Please write your answer on the next page.

**Answer to Question 5**

**Algorithm**:

1. Let G' be the graph produced from G by removing all the edges in $out(a)$.
2. Run Dijkstra on G' with $s$ as a source. Denote the output by $d'(s,v)$.
3. Run BFS of G, with $a$ as a source. Denote by $label(a,v)$ the output of this run.
4. For every node v∈V, cost(s,v)=minimum($d'(s,v)$, $d'(s,a) + \frac{label(a,v)(label(a,v)+1)}{2}$).

In steps 1 and 2 we compute for every v∈V the length of a shortest path from s to v that does not pass in $a$ (or only enters $a$ for v= $a$). In step 3, we compute for every v∈V the minimal number of edges in a path from a to v. In step 4, we return for every node the minimum cost between two options: a path that does not pass in $a$, and a path that passes in $a$.

**Correctness**:

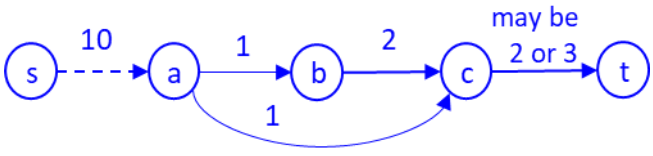
There is a path from s to v with cost x in G ff there is path from s to v in G' that either
(1) is of length x and does not go through $a$
(2) is comprised of a path of length x-(1+2+...+k) to a, and a path with k edges from $a$ to v.

In particular, if the cheapest path passes in $a$, then its cost is d'(s,a)+ (1+2+..+label(a,v)), and if the cheapest path does not pass in $a$ then its cost is d'(s,v).

**Time complexity:**
Step 1: O(|V|+|E|). G' has |V| nodes and at most |E| edges.
Step 2: O(|E|+|V|log|V|) assuming implementation of Dijkstra with Fib. heaps.
Step 3: O(|V|+|E|)
Step 4: O(|V|)

All together: O(|E|+|V|log|V|)

**Common mistake:** Run BFS from $a$, make two copies of the graph where the second copy is reached by passing in a. In the second copy, the weight of (u,v) is label(v) in BFS(a). Then run Dijkstra($s_0$). The problem with this solution is that there may be several ways to reach a node, each of them by a path with a different number of edges. For example, in this graph, the edge (ct) may be second or third in a path that leaves a). The path s-a-b-c-t has cost 10+1+2+3 while in your construction, since label(t)=2, it has cost 10+1+2+2.




There were several variants of this mistake. In most of them, the shortest path is computed correctly, so no points were reduced in the 'algorithm' part. However, most of the students who used this approach wrote a wrong statement in the 'correctness' part (and lost 3-4 points, depending on the formulation of the claim). For example, it is not true that there is a path with 3 edges after a, iff the graph you built has a path whose cost is d(s,a)+(1+2+3).

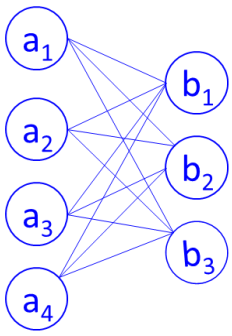
**Question 6 (2*6=12 pts.)**

In each of the following two sections, you have to describe a **simple, connected, undirected, bipartite** graph $G = (A \cup B, E)$ that satisfies a certain property, and has a **maximum number of edges**. We denote by $a_1, a_2, \ldots, a_{|A|}$ and $b_1, b_2, \ldots, b_{|B|}$ the nodes of $A$ and $B$.

- Write down the number of edges in the graph as a function of $|A|$ and$|B|$.
- Explain why this is the maximum number of edges in a simple connected bipartite graph that satisfies the property.
- Describe a corresponding graph in a figure (draw a graph for which $|A| = 4, |B| = 3$)

♣ **6.1** In a run of **BFS($a_1$)**, a directed spanning tree is obtained where $a_2, \ldots, a_{|A|}$ are leaves (nodes with out-degree=0). There may be additional leaves in the tree.

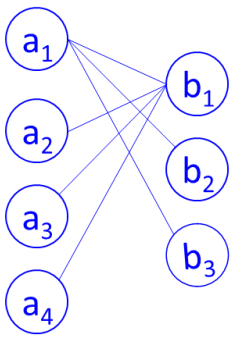
The number of edges is |A|·|B|. This is the maximal number since G is simple. All the nodes in $a_2, \ldots, a_{|A|}$ are leaves, since they are all labeled '2' and all the nodes in B are labeled '1'.




♣ **6.2** In a run of **DFS($a_1$)**, a directed spanning tree is obtained where $a_2, \ldots, a_{|A|}$ are leaves (nodes with out-degree=0). There may be additional leaves in the tree.

The number of edges is |A|+|B|-1. This is the maximal number since G must be a tree.
Assume by contradiction that G includes a cycle, and let $a_k$ be the first among $a_2, \ldots, a_{|A|}$ that belongs to a cycle and is marked by DFS. When $a_k$ is marked, it has at least one unmarked neighbor in B (by the cycle, it has at least two neighbors), contradicting the property that it is a leaf in the DFS tree.

(or any other tree)




Common mistake: it is not sufficient to explain why a specific graph, that you provide has a maximal number of edges. Your justification should cover any graph with |A|+|B|-1 edges.